Documentación del proyecto

Se debe realizar la siguiente documentación por cada proyecto.

Fase 1. Contextualización

- ¿Qué se debe hacer?

Desarrollar un programa que simule y obtenga la ruta más corta para visitar los Pueblos Mágicos de un estado de México. México cuenta con 177 Pueblos Mágicos en los 31 estados, y cada estado tiene entre 2 y 12 conectados por carreteras con distancias conocidas.

Se modelan los Pueblos Mágicos de un estado (Puebla, Sierra Norte) como un grafo ponderado. El usuario introduce un pueblo de origen y uno de destino, y el sistema calcula y muestra la ruta óptima (la de menor distancia total en kilómetros) usando uno de los tres algoritmos clásicos de caminos mínimos. Se eligió Dijkstra y se justifica su elección:

- Las distancias por carretera son siempre positivas, por lo que no se requiere soporte de pesos negativos (ventaja de Bellman-Ford y Floyd-Warshall).
- La consulta es de un origen a un destino (origen único); Floyd-Warshall calcularía las distancias entre todos los pares ($O(V^3)$) sin necesidad.
- Con cola de prioridad su complejidad es $O((V + E) \log V)$, más eficiente que Bellman-Ford ($O(V \cdot E)$) en un grafo disperso.
- Se puede detener en cuanto el destino se extrae de la cola, y con el vértice previo se reconstruye la ruta exacta.

- ¿Qué se necesita?

Datos	Pueblos Mágicos del estado y distancias por carretera entre los que se conectan directamente (enunciado, Google Maps o fuentes oficiales).
Modelo	Grafo ponderado no dirigido representado con lista de adyacencia (grafo disperso: 10 aristas de 21 posibles).
Archivo de datos	Archivo de texto con una arista por línea: PuebloA PuebloB km; comentarios con #.
Algoritmo	Dijkstra con cola de prioridad (heapq) y reconstrucción de ruta con vértices previos.

Interacción	Menú de consola para elegir origen y destino, validar la entrada y repetir consultas.
Salida	Ruta tramo por tramo, ruta completa y distancia total en km.
Herramientas	Python 3 (módulo heapq), editor de código y Graphviz para el diagrama UML.

- ¿Qué elementos se identifican?

Elemento	Rol en el problema	Representación
Pueblo Mágico	Vértice (nodo)	Clase PuebloMagico
Carretera	Arista con peso (km)	Clase Carretera
Red de carreteras	Grafo no dirigido	Clase Grafo (lista de adyacencia)
Pueblo de origen / destino	Datos de entrada del usuario	Opciones 1 a 7 del menú
Algoritmo de camino mínimo	Lógica de solución	Clases AlgoritmoRutaMinima y Dijkstra
Ruta y distancia total	Resultado	Clase ResultadoRuta
Menú de consola	Interfaz con el usuario	Clase RutaMagica

Chiapas

#	Pueblo Mágico	Clave
1	Chiapa de Corzo	CdC
2	Comitán de Domínguez	CdD
3	Copainalá	COP
4	Ocozocoautla de Espinosa	OE
5	Palenque	PAL
6	San Cristóbal de las casas	SCdC

Matriz de Adyacencia de Referencia (distancias en km)

Nota: El símbolo ∞ indica que no existe conexión directa entre esos dos pueblos.

	CdC	CdD	COP	OE	PAL	SCdC
CdC	0	∞	84.8	49	∞	54.1
CdD	∞	0	∞	∞	∞	90.3
COP	84.1	∞	0	83.5	∞	∞
OE	48.1	∞	83.5	0	∞	∞
PAL	∞	∞	∞	∞	0	213
SCdC	54.1	90	∞	∞	213	0

Fase 2. Definición de los requerimientos funcionales

Se debe elaborar el listado de los requerimientos del contexto planteado

Requerimientos	
R1	Cargar el grafo de Pueblos Mágicos desde un archivo de texto
R2	Mostrar la lista de Pueblos Mágicos disponibles
R3	Seleccionar pueblo de origen y pueblo de destino
R4	Calcular la ruta más corta con el algoritmo de Dijkstra
R5	Mostrar la ruta encontrada y la distancia total
R6	Manejar casos especiales (mismo pueblo, sin ruta)
R7	Consultar otra ruta sin reiniciar el programa

Para la documentación de cada requerimiento funcional utilizar el siguiente esquema descriptivo.

Nombre	R1 – Cargar el grafo desde archivo
Descripción	Lee el archivo de texto línea por línea ignorando líneas vacías y comentarios (#). Cada línea define una carretera entre dos pueblos con su distancia en km. Registra cada carretera en ambos sentidos para formar un grafo no dirigido con lista de adyacencia.
Entrada	Archivo pueblos_chiapas.txt con líneas de formato: PuebloA PuebloB distancia_km.
Salidas	Grafo en memoria con 6 vértices y 6 aristas (12 entradas de adyacencia).

Nombre	R2 – Mostrar la lista de Pueblos Mágicos
Descripción	Presenta el encabezado del programa y la lista numerada (1 a 6) con el nombre completo de cada Pueblo Mágico del estado.
Entrada	Ninguna (usa el grafo cargado).
Salidas	Menú en pantalla con los pueblos numerados.

Nombre	R3 – Seleccionar origen y destino
Descripción	Solicita al usuario el número del pueblo de origen y el del pueblo de destino. Valida que la entrada sea un entero dentro del rango 1 a 6 y repite la solicitud mientras sea inválida.
Entrada	Dos números enteros entre 1 y 6 (origen y destino).
Salidas	Los objetos PuebloMagico seleccionados y la confirmación en pantalla (-> Origen / -> Destino).

Nombre	R4 – Calcular la ruta más corta (Dijkstra)
Descripción	Aplica Dijkstra con cola de prioridad: inicia en 0 la distancia del origen e infinito en el resto, extrae siempre el vértice más cercano y relaja sus aristas (si d + peso < distancia del vecino, la actualiza y guarda el predecesor). Se detiene al llegar al destino y reconstruye la ruta siguiendo los predecesores.
Entrada	Grafo, pueblo de origen y pueblo de destino.
Salidas	Objeto ResultadoRuta con la lista ordenada de pueblos y la distancia total en km.

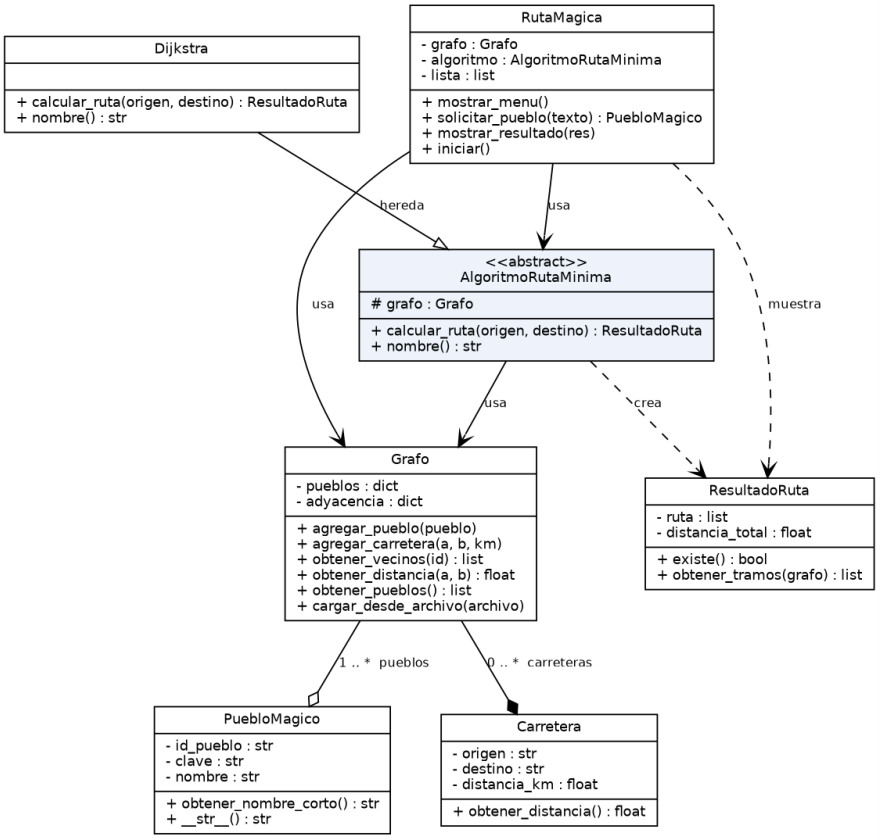
Nombre	R5 – Mostrar la ruta y la distancia total
Descripción	Imprime cada tramo con su distancia (Origen --(km)--> Destino), la ruta completa separada por flechas y la distancia total en kilómetros, con el formato del ejemplo de ejecución del enunciado (adaptado a Chiapas).
Entrada	Objeto ResultadoRuta.
Salidas	Mensaje [OK] RUTA ENCONTRADA, tramos, ruta completa y distancia total.

Nombre	R6 – Manejar casos especiales
Descripción	Si el origen es igual al destino, la ruta es ese único pueblo con 0 km. Si no existe camino entre los pueblos, informa que no hay ruta en lugar de fallar.
Entrada	Pueblo de origen y pueblo de destino.
Salidas	Ruta de un solo pueblo con 0 km, o el mensaje [X] No existe ruta entre esos pueblos.

Nombre	R7 – Consultar otra ruta
Descripción	Después de cada resultado pregunta al usuario si desea otra consulta. Si responde S (mayúscula o minúscula) vuelve a pedir origen y destino; con cualquier otra respuesta termina el programa.
Entrada	Respuesta S/N.
Salidas	Nueva consulta o fin del programa.

Fase 3. Diseño

Determinar las clases que conforman el sistema, junto con los atributos y los métodos y crear el diagrama de clases UML, así como sus relaciones de herencia (si aplica).



Fase 4. Codificación

Se deben escribir los programas fuentes necesarios, basados en el diagrama de clase de la fase anterior:

- Implementación de los métodos (construcción de la parte lógica).
- Codificación de fórmulas
- Asignaciones
- Condicionales
- Documentación del código

Fase 5. Pruebas.

Aquí se debe:

- Verificar que el programa cumpla con los requerimientos establecidos.
- Construcción de los documentos de entrega final.
- Adjuntar evidencias de resultados de la ejecución (impresión de pantallas).